

Gymnázium, Praha 6, Arabská 14

vyučující Mgr. Jan Lána

VirtBank

Lukáš Hrtan, Richard Toman a Eliška Preclíková

3.E



6. března 2026

Poděkování

Chtěli bychom poděkovat našim rodičům za pomoc s nastavením databáze, za jejich trpělivost a pomoc při řešení problémů, které se během práce vyskytly.

Dále bychom chtěli poděkovat našemu vedoucímu práce, Mgr. Janu Lánovi, za jeho vedení. V neposlední řadě patří naše poděkování paní ředitelce RNDr. Zdeňce Hamhalterové za snahu při řešení kvalifikovaného certifikátu (i přes nepříznivý výsledek) a také technické podpoře API Komerční banky, která nám pomohla s pochopením komunikace s bankovním API.

Anotace

Cílem tohoto projektu je vývoj webové aplikace VirtBank, která slouží ke sjednocené správě více bankovních účtů v rámci jednoho rozhraní. Aplikace využívá moderní bankovní API standardy pro bezpečný import dat a jejich následnou vizualizaci. Důraz je kladen na přehlednost, možnost kategorizace transakcí a podporu finanční gramotnosti uživatelů. Back-end řešení je postaveno na frameworku Spring Boot, zatímco front-end je optimalizován pro desktopové zobrazení s ohledem na specifické potřeby rodinného rozpočtování.

The goal of this project is to develop a web application, VirtBank, designed for the unified management of multiple bank accounts within a single interface. The application utilizes modern banking API standards for secure data import and subsequent visualization. Emphasis is placed on clarity, the ability to categorize transactions, and supporting users' financial literacy. The back-end solution is built on the Spring Boot framework, while the front-end is optimized for desktop viewing, taking into account the specific needs of family budgeting.

Das Ziel dieses Projekts ist die Entwicklung einer Webanwendung namens VirtBank, die zur zentralen Verwaltung mehrerer Bankkonten innerhalb einer einzigen Benutzeroberfläche dient. Die Anwendung nutzt moderne Banking-API-Standards für den sicheren Datenimport und die anschließende Visualisierung. Besonderer Wert wird auf Übersichtlichkeit, die Möglichkeit der Transaktionskategorisierung und die Förderung der Finanzkompetenz der Nutzer gelegt. Die Backend-Lösung basiert auf dem Spring Boot Framework, während das Frontend für die Desktop-Ansicht optimiert ist, wobei die spezifischen Anforderungen der Haushaltsplanung berücksichtigt wurden.

Prohlášení

Prohlašujeme, že jsme ročníkovou práci vypracovali samostatně pod vedením Mgr. Jana Lány a že jsme uvedli všechny použité informační zdroje a literaturu v souladu se zásadami citování.

V Praze dne:

Vlastnoruční podpisy autorů:

Lukáš Hrtan:

Richard Toman:

Eliška Preclíková:

Obsah

1. Úvod.....	1
1.1. Cíl práce.....	1
1.2. Motivace a přínos projektu.....	1
2. Komplikace.....	2
2.1. Legislativní a technické aspekty získávání dat.....	2
2.2. Implementace bankovních API.....	2
3. Použité moduly a třídy (Back-end).....	3
3.1. Framework Spring Boot.....	3
3.1.1. PostgreSQL Driver (Databáze).....	3
3.1.2. Spring Security (Základní ochrana).....	3
3.1.3. Spring Web.....	3
3.1.4. Dependency Injection.....	3
3.2. Datové zpracování a mapování.....	4
3.3. Správa konfigurací (Properties).....	4
4. Komunikace s bankou.....	5
4.1. Registrace aplikace na webu API.....	5
4.2. Autorizace uživatelů.....	5
4.3. Provádění dotazů.....	5
5. Front-end naší aplikace.....	6
5.1. Manipulace s více účty najednou.....	6
5.2. Vizualizace a klasifikace transakcí.....	6
6. Ergonomie a architektura klientského rozhraní.....	7
6.1. Pokročilá správa multikontních struktur.....	7
6.2. Mechanismus synchronizované komparace transakcí.....	7
6.3. Uživatelsky definovaná taxonomie a sémantické vrstvy.....	8
6.4. Inteligentní interpretace a vizualizace aktiv.....	8
6.5. Dynamické renderování analytických statistik.....	9
7. Ukázky kódu.....	10
7.1. Načítání Property hodnot.....	10
8. Do budoucna.....	11
Zdroje.....	12

1. Úvod

1.1. Cíl práce

Primárním cílem této práce bylo navrhnout a implementovat webovou platformu, která by fungovala jako centrální správce různých bankovních účtů. Naše aplikace umožňuje tyto účty spravovat na jednom místě, čímž zvyšuje efektivitu kontroly nad osobními či rodinnými financemi.

V rámci vývoje jsme se zaměřili na vytvoření dvou prostředí:

1.1.1. Produkční prostředí

Určené pro napojení reálných bankovních dat.

1.1.2. Sandbox

Izolované testovací prostředí sloužící pro vývoj a simulaci transakcí bez rizika manipulace s reálnými prostředky.

1.2. Motivace a přínos projektu

Výběr tématu byl motivován snahou lépe porozumět mechanismům moderního bankovníctví a tokům peněz v digitálním prostoru. Součástí naší vize bylo také začlenění vzdělávací, nebo informační složky zaměřené na finanční gramotnost.

2. Komplikace

2.1. Legislativní a technické aspekty získávání dat

Získávání dat z bankovních účtů představuje pro vývojáře třetích stran značnou výzvu. Přímé stahování dat bez oficiálního rozhraní (tzv. screen scraping) je v mnoha ohledech problematické a může způsobit legislativní potíže. Proto jsme zvolili cestu využití oficiálních bankovních API (Application Programming Interface).

2.2. Implementace bankovních API

Původním záměrem bylo využití API Komerční banky. Narazili jsme však na problém vlastnit kvalifikovaný certifikát pro přístup do produkční části. Po konzultaci s vedením školy a prozkoumání dostupných možností jsme se rozhodli změnu na API České spořitelny. Tato banka nabízí velmi propracované sandbox prostředí, které umožňuje simulovat reálné funkce banky bez nutnosti okamžité certifikace, což bylo pro potřeby školního projektu ideální.

3. Použité moduly a třídy (Back-end)

3.1. Framework Spring Boot

Jako základ pro vývoj backendu byl zvolen framework Spring Boot. Tato volba je především z jeho prověřenosti a velké podpory.

3.1.1. PostgreSQL Driver (Databáze)

Framework disponuje robustními moduly pro práci s databázemi (např. Spring Data JPA), které automatizují běžné operace a zajišťují bezpečnost dat.

3.1.2. Spring Security (Základní ochrana)

Integrované bezpečnostní protokoly umožňují snadnou implementaci autorizace a autentizace, což minimalizuje riziko vzniku bezpečnostních děr.

3.1.3. Spring Web

Integrovaný webový server, což urychlilo proces nasazení.

3.1.4. Dependency Injection

Abychom eliminovali memory leak a dodrželi principy čistého kódu, využíváme Dependency Injection (DI) od Springu. Ten v rámci Component Scanu automaticky vyhledává třídy s anotacemi `@Component`, `@Service`, `@Controller` či `@RestController`. Kromě toho zpracovává i `@Configuration` třídy, kde jsou beany definovány ručně pomocí metod označených anotací `@Bean`.

Všechny tyto objekty Spring zaregistruje do svého kontejneru jako tzv. Beany. Ve výchozím nastavení jsou typu Singleton, což znamená, že v celé aplikaci existuje pouze jedna sdílená instance. Pro vkládání těchto závislostí preferujeme konstruktor, které zajišťuje mít proměnné jako final, ale Spring podporuje i vkládání přes anotaci `@Autowired`.

3.2. Datové zpracování a mapování

Vzhledem k tomu, že banky komunikují primárně prostřednictvím formátu JSON, využili jsme knihovnu Jackson (Object Mapper). Tato třída zajišťuje automatizované mapování JSON odpovědí na předem definované Java objekty, což minimalizuje chybovost při manipulaci s daty.

3.3. Správa konfigurací (Properties)

Pro efektivní správu cest k různým bankovním API jsme implementovali systém založený na konfiguračních třídách. Abychom zachovali kód srozumitelný a modulární, vytvořili jsme vlastní architekturu, kde každá banka disponuje vlastní instancí konfigurační třídy. To umožňuje snadné rozšiřování aplikace o další banky v budoucnu.

4. Komunikace s bankou

Komunikace mezi naší aplikací a bankovními servery probíhá standardně pomocí HTTP requestů. Klíčové informace jsou přenášeny v hlavičkách (headers), query parametrech nebo v těle požadavku.

4.1. Registrace aplikace na webu API

Pro realizaci komunikace jsou nezbytné specifické identifikační údaje, jejichž způsob se u jednotlivých bankovních institucí liší. Společným jmenovatelem je však proces registrace aplikace na vývojářském portálu dané banky. Výsledkem této registrace je zpravidla přidělení unikátního identifikátoru aplikace (*Client ID*) a tajného klíče (*API Key* nebo *Client Secret*).

4.2. Autorizace uživatelů

Samotná registrace aplikace v API portálu neumožňuje automatický přístup k uživatelským účtům. K tomuto účelu slouží protokol OAuth 2.0, který definuje mechanismus, jakým aplikace získává omezený přístup k datům jménem uživatele. Proces standardně probíhá přesměrováním uživatele na přihlašovací stránku serveru, po jehož úspěšném ověření je vrácen autorizační kód (code). Tento kód následně slouží k získání *access_token* a *refresh_token*.

Access_token Slouží k přímé komunikaci se serverem. Má omezenou dobu platnosti obvykle v řádu jednotek až desítek minut.

Refresh_token Slouží k obnovení expirovaného *access_token* bez nutnosti opětovné interakce uživatele. Jeho platnost je dlouhodobá v řádu měsíců až jednoho roku.

4.3. Provádění dotazů

Na základě dokumentace k API jsou stanoveny potřebné parametry pro jednotlivé požadavky, jako jsou endpointy, query parameters, hlavičky a struktura těla požadavku. Dokumentace rovněž definuje formát očekávané odpovědi, což je nejčastěji datová struktura ve formátu JSON, nebo výčet chybových kódů a jejich významů. V naší aplikaci byly primární dotazy směřující k získání informací o bankovních účtech a platebních kartách.

5. Front-end naší aplikace

Aplikace je primárně navržena pro desktopová zařízení, protože správa rodinných financí vyžaduje větší plochu pro zobrazení více účtů, což je na mobilních zařízeních často nepřehledné a mimo domácnost se to nehodí.

5.1. Manipulace s více účty najednou

Uživatelské rozhraní aplikace je navrženo pro efektivní správu více bankovních účtů současně na jednom místě, kde jsou jednotlivé účty uspořádány v buňkách vedle sebe a vizuálně simulují prostředí dané banky. Klíčovým prvkem pro usnadnění porovnávání dat je funkce synchronizovaného posunu, která po „uzamknutí“ pohledu umožňuje vertikálně scrollovat transakční historií u všech propojených účtů najednou, což je ideální pro rychlé porovnávání výdajů v čase. Pro jednodušší porovnání si uživatel může pořadí zobrazených účtů libovolně měnit.

5.2. Vizualizace a klasifikace transakcí

Aplikace umožňuje uživateli definovat vlastní kategorie pro příjmy a výdaje. Na základě těchto dat systém generuje grafické přehledy a statistiky podle kategorií i celkových výdajů. Pro zvýšení přehlednosti jsou pak jednotlivé transakce v seznamu barevně odlišeny.

6. Ergonomie a architektura klientského rozhraní

Projekt VirtBank se vědomě odchyluje od současného trendu „mobile-first“ a je primárně optimalizován pro pracovní stanice a desktopové prohlížeče. Tato volba vychází z předpokladu, že efektivní správa komplexních rodinných aktiv vyžaduje vizuální stabilitu a široký horizontální prostor, který mobilní terminály neposkytují. Desktopové rozhraní eliminuje nutnost agresivní komprese dat, což uživateli umožňuje vnímat souvislosti mezi více účty bez nutnosti přepínání kontextu.

V rámci domácího rozpočtování funguje jako analytický terminál. Velká zobrazovací plocha je využita k paralelnímu vykreslování historických trendů a transakčních seznamů, což snižuje kognitivní zátěž uživatele. Ten tak může v reálném čase sledovat tok financí v širším kontextu, což je v prostředí mimo domov či na malém displeji technicky i prakticky nerealizovatelné.

6.1. Pokročilá správa multikontních struktur

Rozhraní aplikace je postaveno na adaptivní modulární mřížce (Grid System). Každá bankovní entita je zapouzdřená do samostatného vizuálního kontejneru, který skrze CSS parametry adoptuje estetické prvky konkrétní bankovní instituce. Tento přístup vytváří digitální simulaci reálného bankovního prostředí, ve kterém se uživatel orientuje na základě barevné a tvarové asociace.

Zásadním inovativním prvkem je možnost uživatelské rekonfigurace gridu. Pomocí asynchronního přeuspořádání komponent si může správce financí nastavit hierarchii účtů podle aktuální priority. Tím se dosahuje stavu, kdy nejdůležitější bilanční údaje (např. hlavní cash-flow) jsou vždy v primárním zorném poli, zatímco sekundární nebo termínované účty jsou umístěny v analytické periférii.

6.2. Mechanismus synchronizované komparace transakcí

Pro potřeby hloubkového auditu výdajů byla vyvinuta unikátní funkce „Zámek časové osy“. Na rozdíl od běžných bankovních aplikací, kde uživatel

kontroluje každý účet izolovaně, Virtbank umožňuje propojení vertikálních scrollovacích událostí u všech zobrazených entit.

Z technického hlediska jde o zachytávání **wheel** a **touch** eventů, které jsou následně redistribuovány mezi všechny viditelné kontejnery. To dovoluje uživateli provádět vertikální řez historií – například vidět přesný moment odchodu nájmu z jednoho účtu a současný přísun úroků na účet jiný. Tato schopnost komparace dat v identickém časovém bodě je klíčová pro pochopení vnitřní cirkulace peněz v rámci rodiny nebo malého podniku.

6.3. Uživatelsky definovaná taxonomie a sémantické vrstvy

Systém nevyužívá pouze pevné bankovní kategorie, které jsou často nepřesné, ale umožňuje uživateli definovat vlastní sémantické vrstvy (custom categories). Tyto vrstvy jsou na transakce aplikovány skrze mapovací logiku na straně front-endu. Tento proces transformuje prostý seznam plateb na strukturovaný datový model vhodný pro statistické zpracování.

Každé kategorii lze přiřadit specifický sémantický význam, který se následně propisuje do všech analytických modulů. Uživatel tak může oddělit běžné provozní náklady od mimořádných investic, což poskytuje mnohem realističtější obraz o disponibilním zůstatku.

6.4. Inteligentní interpretace a vizualizace aktiv

Vizuální hierarchie transakcí je podpořena barevným kódováním na základě finančního vektoru (příjem vs. výdaj) a typu kategorie.

- Zelené barevné spektrum signalizuje kladné saldo a příchozí toky.
- Spektrum teplých barev (červená/oranžová) je vyhrazeno pro odchozí transakce.
- Neutrální tóny se využívají pro vnitřní převody mezi vlastními účty, aby nezkreslovaly vnímanou bilanci.

Tento barevný systém umožňuje tzv. „visual skimming“ – schopnost uživatele během zlomku sekundy odhadnout poměr mezi příjmy a výdaji v dlouhém seznamu bez nutnosti čtení konkrétních numerických hodnot.

6.5. Dynamické renderování analytických statistik

Vrcholnou vrstvu tvoří asynchronní grafické moduly. Na rozdíl od statických obrázků jsou tyto grafy přímo napojeny na filtrovací logiku rozhraní. Při změně zobrazeného období nebo deaktivaci určité kategorie v legendě dochází k okamžitému přepočtu datových sad a re-renderování grafu pomocí knihovny Chart.js.

Uživatel má k dispozici lineární grafy pro sledování vývoje čistého jmění (Net Worth) a prstencové grafy pro strukturální analýzu výdajů. Tyto vizualizace neslouží pouze k zobrazení historie, ale díky propojení s celým systémem Virtbank umožňují identifikovat nadbytečné výdaje a optimalizovat budoucí rozpočtové plány.

7. Ukázky kódu

7.1. Načítání Property hodnot

```
public enum Bank { CS, KB } // Seznam implementovaných bank.
public enum Environment { SANDBOX, PRODUCTION } // Prostředí
private final Dictionary<String, Properties> props; // Objekt pro data z .properties souborů.
private final static String customPropsKey = "API"; // Klíč pro data naší aplikace.
private final static String[][] files = {
    {"custom.properties"},
    {"cs.properties", "cs.secret.properties"},
    {"kb.properties"}
}; // Názvy finálních souborů. 1. pole je pro aplikaci, následující jsou seřazeny podle Bank
// enumu.
public Property() { // Konstruktor.
    props = new Hashtable<>();
    for (int i = 0; i < files.length; i++) {
        String key;
        if (i == 0) { // Vytvoření nového key-value páru do props.
            key = customPropsKey;
            props.put(key, new Properties());
        } else {
            key = Bank.values()[i - 1].name(); // Získání názvu banky.
            props.put(key, new Properties());
        }
        for (String file : files[i]) { // Načtení všech souborů.
            try (InputStream in = getClass().getClassLoader().getResourceAsStream(file)) {
                // Získání cesty ke složce "resources" aplikace a jejímu podsouboru file.
                props.get(key).load(in); // Samotné načtení souboru do objektu.
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        }
    }
}
```

8. Do budoucna

V budoucnu plánujeme rozšíření o mobilní aplikaci, která by však nesloužila jako hlavní analytický nástroj, ale spíše jako rychlý přehled stavu účtů na cestách. Dalším krokem by mohl být vývoj algoritmů pro automatickou klasifikaci plateb pomocí strojového učení, čímž by se dále zvýšil uživatelský komfort.

Zdroje

Použité dokumentace:

Tailwind CSS, 17 June 2021, <https://v2.tailwindcss.com/docs>. Accessed 4 March 2026.

Spring Initializr, <https://start.spring.io/>. Accessed 4 March 2026.

Erste Group Bank AG. "CSAS." *Erste API*, Česká Spořitelna,
<https://developers.erstegroup.com/docs/apis/bank.csas>.

"KB API." *KB API Developer Portal*, <https://developers.kb.cz/>. Accessed 4 March 2026.

"Spring Boot." *Spring*, <https://docs.spring.io/spring-boot/index.html>. Accessed 4 March 2026.